

Author

Name : Chandramohan

Roll No. : 24f1001605

Student E-mail : 24f1001605@ds.study.iitm.ac.in

About me:

I am a student currently pursuing B.Tech CSE and BS DS passionate about software development interested in full-stack web development, mobile app development and automation. I enjoy building practical solutions to real-world problems and learning new technologies.

Description

This project implements a web-based vehicle parking management system with separate dashboards and features for admins and users. The system allows admins to manage parking lots and users to book, occupy, and release parking spots, with cost calculation and analytics.

Technologies used

- Python 3.10
- Flask (Web framework)
- Flask-Login (User authentication and session management)
- Flask-SQLAlchemy (ORM for database interaction)
- Flask-WTF (Form handling and validation)
- Jinja2 (HTML templating)
- SQLite (Database)
- Bootstrap (Frontend styling)
- Chart.js (Frontend analytics)
- WTForms, email_validator, Werkzeug, itsdangerous, MarkupSafe (supporting libraries)

Flask provides a lightweight, modular web framework. Flask-Login and Flask-WTF simplify authentication and form handling. Flask-SQLAlchemy abstracts SQL queries and manages relationships. Jinja2 enables dynamic HTML rendering. Bootstrap and Chart.js enhance the user interface and data visualization. SQLite is used for its simplicity and ease of setup for small projects.

DB Schema Design

- User:
 - id (PK, int, auto-increment)
 - username (str, unique, not null)
 - password_hash (str, not null)
 - role (str, default 'user')
 - created_at (datetime, default now)
- ParkingLot:
 - id (PK, int, auto-increment)
 - prime_location_name (str, not null)
 - price_per_hour (float, not null)
 - address (str, not null)
 - pincode (str, not null)
 - max_spots (int, not null)
 - created_at (datetime, default now)
- ParkingSpot:
 - id (PK, int, auto-increment)
 - lot_id (FK to ParkingLot, int, not null)
 - status (str, default 'A')
 - spot_number (int, not null)

- Reservation:
 - id (PK, int, auto-increment)
 - spot_id (FK to ParkingSpot, int, not null)
 - user_id (FK to User, int, not null)
 - parked_at (datetime, default now)
 - released_at (datetime, nullable)
 - total_cost (float, nullable)

The schema is normalized for clarity and efficiency. Each entity (user, lot, spot, reservation) is separated for scalability. Relationships are managed via foreign keys, and constraints ensure data integrity (e.g., unique usernames, only one active reservation per user).

API Design

APIs are provided for:

- Listing all parking lots (GET /api/lots)
- Getting details of a specific spot (GET /api/spots/<id>)
- Fetching the current user's reservations (GET /api/reservations/me, login required)

APIs are implemented using Flask routes, returning JSON responses. Authentication is enforced where needed. The API design is RESTful, making it easy to integrate with other systems or frontend AJAX calls.

Architecture and Features

The project follows a modular MVC-like structure.

- Controllers: All business logic is in the controllers/ directory, separated by admin, user, and authentication concerns.
- Models: Database models are in models/models.py.
- Templates: HTML templates are organized in templates/ by user role (admin, user, auth).
- Static files: CSS and JS are in static/.
- Configuration: App settings are in config.py.
- Database initialization: Handled by init_db.py.

Features Implemented

- Admin: Can add, edit, and delete parking lots (with auto-generated spots), view dashboards with occupancy and revenue analytics, search by spot ID, and manage users.
- User: Can register, log in, book the first available spot in a lot, release spots (with cost calculation), and view booking history and analytics.
- Security: Passwords are hashed, and role-based access is enforced.
- REST API: Minimal endpoints for lots, spots, and reservations.
- Frontend: Responsive UI with Bootstrap and dynamic charts using Chart.js.

Additional Features

- Admin can only delete lots if all spots are free.
- Users can have only one active reservation at a time.
- Analytics and charts for both admin and user dashboards.

Video Demo

https://drive.google.com/file/d/1D-3Nf_TxS_cMz0N6yLzwGO6856VGXEKP/view?usp=sharing